VisionPong
Aneesh Pabolu
5/28/2026
Yilmaz - Period 3

Table of Contents

Abstract……3
I. Introduction……...3
II. Background……..4
III. Applications……4
IV. Methods……...5
V. Results & Limitations.……7
VI. Conclusion.……7
VII. Future Work and Recommendations……....8
VIII. Issues, Fixes and Advice……..……...……..……...8
References.……...10

Abstract

There have been many motion-tracking video games where a digital avatar moves based on a player's physical movement in the real world. These games work by taking information from physical controllers or sensors. Common games include Kinect Sports, 1-2 Switch, and Mario Party. However, motion-controlled games typically require expensive hardware such as game consoles such as an Xbox console or Nintendo Switch. Furthermore, "vision" based motion controlled games have practically become obsolete. In this project, I try to bring back "vision" based motion-controlled games by developing a framework for these video games to run directly off your computer and track your movement using your webcam. In this project, I take inspiration from the Xbox Kinect Sports Table Tennis game and try to replicate it using no extra hardware except a computer and a live video feed from a webcam. Running these games locally on your computer can increase the access to motion-controlled games and improve research relating to vision based digital avatar control. To replicate human behavior in a digital game space, I utilized the MediaPipe Pose estimation model to receive Normalize Image Coordinates and wrote C# scripts to try to replicate the positions of 13 important body landmarks in the Unity game development engine. This process successfully detected relative human limb movement and horizontal location of a human body. However, translating, perspective-dependent camera observations into proper depth perception, raised additional challenges.

I. Introduction

In this paper, I explore an approach to promote accessibility to vision based motion-controlled games by developing a framework for these games to run on consumer computers that receive information from attached cameras or webcams. As a kid, I grew up playing on the Microsoft developed Xbox Kinect, one of the first most advanced approaches to controllerless motion controlled games. However, Microsoft unfortunately discontinued the Xbox Kinect in 2017 and has since not worked on Kinect technology for consumers. With an interest in bringing back "vision" based motion controlled games, I decided to explore a computer vision based approach for these games to run on the computers people already own. This approach would prevent the need for people to purchase extra hardware to play motion-controlled games.

II. Background

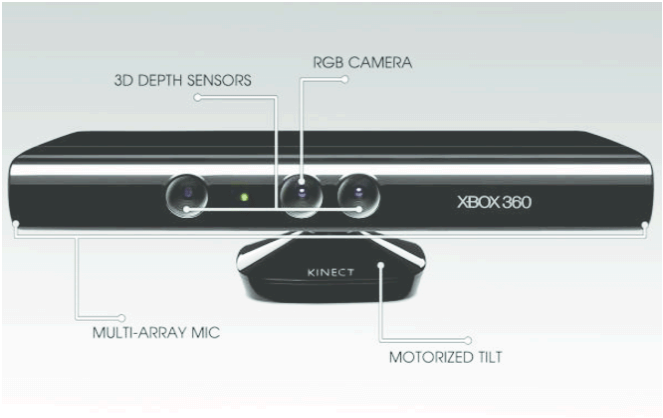


Fig. 1. Xbox 360 Kinect Sensor

To better understand this paper, it is important to understand what motion-controlled games are and the two different styles of motion-controlled games. Motion-controlled games are games in which the main character or digital avatar in the game is controlled by either a part or the whole by a human physically moving in the real world. In a simple sense, there are two main motion-controlled games that have gained streamline popularity: 1.) sensor based motion-controlled games and 2.) “vision” based motion-controlled games. Sensor based motion-controlled games are games in which a sensor almost “feels” human motion as a human moves around a room. Common examples of sensor based motion controlled games include *1 2 Switch* and *Super Mario Party* by Nintendo, which are controlled by Joy-Con’s which have advanced motion capturing sensors to “feel” how a human is moving. A modern and robust example of “vision” based motion controlled games are *Kinect Sports* and *Kinect Adventures*, which are controlled by the Xbox Kinect sensor connected to an Xbox. The Xbox Kinect mainly utilizes Infrared Light emitted from the Xbox Kinect Sensor, refer to Fig. 1, to track the position of the human body in real time. A better understanding of Xbox Kinect, and, in the grander scheme, vision based motion-controlled games can be seen in [2].

III. Applications:

Motion-controlled games are commonly played with a group of friends at a get together as an active way to connect and have fun. They are also extremely important in helping improve access to indoor exercise as they require little to no equipment and a moveable space like a living room. Access to a full body moving avatar can also allow for advanced health metric prediction such as vital movement from the human body and limbs.

IV. Methods:

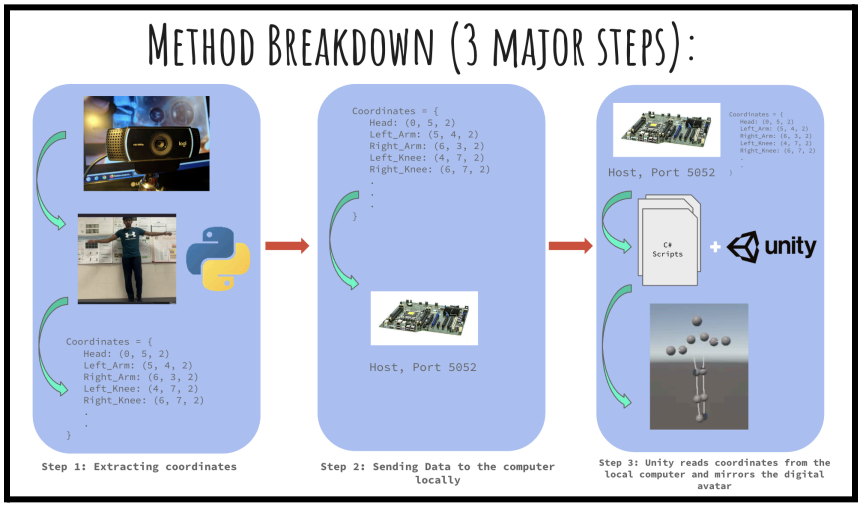


Fig. 2. Method Breakdown

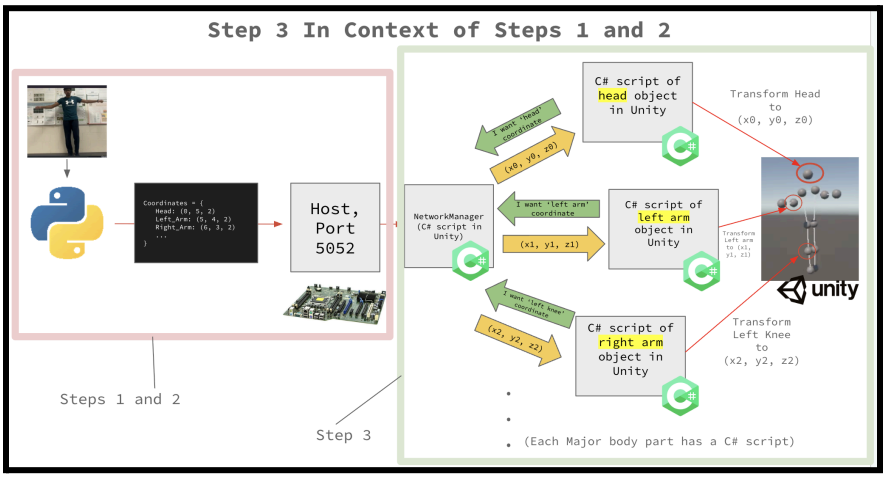


Fig. 4. Step 3 within the context of steps 1 and 2 of the methodology

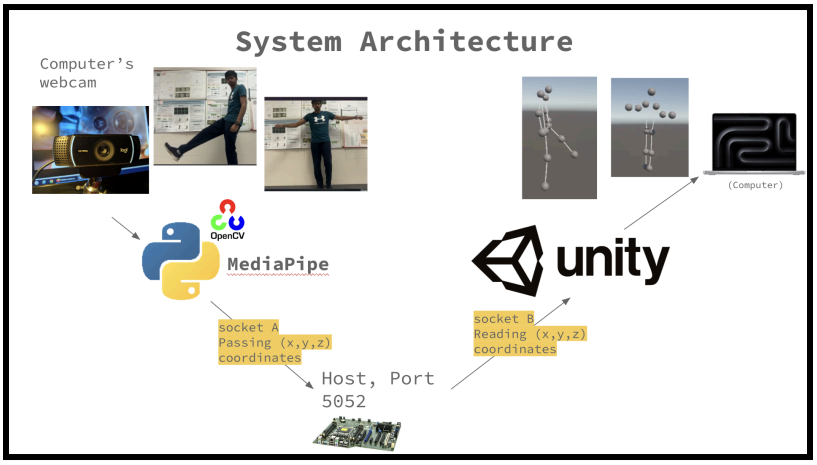


Fig. 5. System Architecture Diagram

At a high level, I took inspiration from Xbox Kinect Sport's Table Tennis game (refer to [3] for an example) and worked on developing a Table Tennis game environment where a digital avatar receives input from the user's webcam to control a digital avatar to play Table Tennis in a digital gamespace. As shown in Fig. 2., my solution involves three major steps: 1.) Extracting coordinates, 2.) sending coordinates to a local port, and 3.) utilizing those coordinates to control a digital avatar in the digital gamespace developed with Unity. A breakdown of this process is as follows.

In step 1, in Python, I utilize OpenCV to capture an image on a frame by frame basis from the default webcam attached to the computer. Then with the MediaPipe Pose Estimation model developed by Google, I extract the locations of 13 key landmarks (nose, right wrist, left wrist, right knee, left knee, right elbow, left elbow, right ankle, left ankle, right shoulder, left shoulder, right hip, left hip) of the human body. Next I convert all provided coordinates into centered coordinates and place the calculated coordinates into a dictionary, which is then processed as JSON text. The Python code sends an updated JSON for each frame captured by OpenCV and processed by MediaPipe. Refer to the "Step 1" section of Fig. 2 for a visual understanding.

In step 2, I send each JSON text to the user's local computer port 5052 through a socket that connects the Python program to the port. This is again done frame by frame. Refer to the "Step 2" section of Fig. 2 for a visual understanding.

Now, the coordinates of each important body landmark have been sent to a local port where external applications can read that information. As shown in Fig. 4, all information sent to the local port is then read into a C# script called NetworkManager. This coordinate information is stored in a public class object in NetworkManager. All important landmarks of the digital avatar in Unity each have their own attached helper C# scripts which interact with the NetworkManager to extract the coordinate components associated with the respective landmark. From here, each helper script transforms the coordinate of the joint it is attached to the updated coordinate received from the public class object in the NetworkManager script. An overview of Step 3 can also be in Fig. 2.

The entire process is highlighted once again in Fig. 5.

V. **Results & Limitations**:

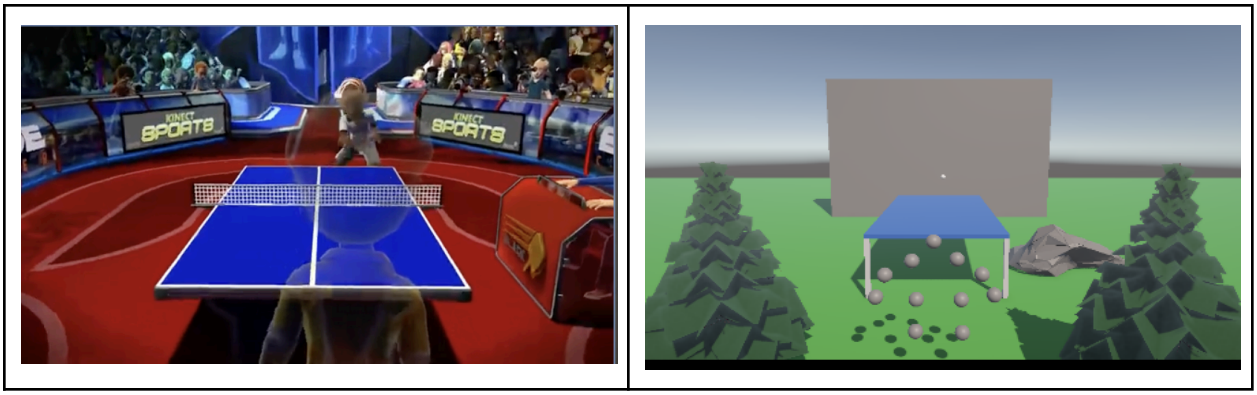


Fig. 6. Xbox Kinect's Kinect Sports Table Tennis game (left) compared with VisionPong's game environment (right)

As shown in [4], my current framework is able to recognize human poses and the proper position of important human landmarks and is able to display them in the Unity game scene with proper accuracy. All horizontal movements of the player and precise location of their limbs are also well replicated in Unity. Basic Physics interactions with game objects have also been implemented, showing feasibility for pursuing this project. A side by side comparison between Microsoft's Xbox Kinect and VisionPong's game environment can be seen in Fig. 6.

Although my framework is able to replicate the horizontal location of the body and the position of limbs in the digital gamespace, a major limitation in sensing the relative depth of the human body with respect to the camera still remains. Currently, due to MediaPipe's Pose Estimation model not working based on 3D depth with real world measurement units, depth is mostly simulated in VisionPong's game environment by a scaling of the digital avatar on a 2D plane. Although this is not an issue in smaller games, it is a current limitation that prevents a full-body motion-controlled game using Computer Vision alone. I explore a potential solution for this problem in the Future Work section of this paper.

VI. **Conclusion**:

This project shows that it is possible to develop "vision" based motion-controlled games using only hardware potential consumers already own. Using the open source MediaPipe Pose Estimation model that can run on laptops and the Unity game development engine where the game I developed can be exported as applications to run on multiple devices like Mac, Windows, and Linux, the project proves how motion-controlled games can become more accessible for all.

VII. **Future Work**:

In order to improve the quality of "vision" based motion-controlled games, my first next step, as hinted in the Results & Limitations section, will be improving depth perception. Because the MediaPipe Pose Estimation model and a majority of other Pose Estimation Models aren't able to provide depth coordinates that are particularly useful for this project, I plan on implementing my own depth perception by analyzing frames and the distance between two major landmarks such as the head and a shoulder. I plan on using the Euclidean distance formula or an alternative for this formula and analyze how the distance between the head and the shoulder changes in consecutive frames. I then plan on multiplying the difference between distance values between different frames with a weighted scalar to find a change in Δz, which I will then add to the previous z value to calculate the change in depth of the object from the camera.

After perfecting distance calculations, I plan on developing a template for developers for Computer Vision based "vision" based motion-controlled games so developers can utilize my project's framework to allow digital avatars in their games be controlled by real human behavior.

VIII. **Issues, Fixes and Advice**:

For people who are interested particularly in this research, I am writing this section to go over some obstacles that I faced throughout this project to prevent you from going through similar problems.

1.) Depth perception: As explained in the Results & Limitations part of my paper, Depth perception is the next biggest obstacle to tackle and a potential solution is presented in the "Future Work" Part of this paper
2.) Landmarks vs. WorldLandmarks: When reading through MediaPipe's documentation for their Pose Estimation Model, you can see that MediaPipe expresses that the Pose Estimation model also returns World coordinates. I tried using World Landmarks, however they didn't work out as I expected for this project and used MediaPipe's default "Landmarks" coordinates.
3.) Learning Unity: I knew next to nothing about game development prior to pursuing this project, which made it a struggle throughout the starting stages of the project (essentially, I tried using Unity without knowing anything about Unity). I don't recommend trying to pursue this project without learning how to actually use the game development you decide to use. If you are also planning to use Unity, I highly recommend going through Unity's own free tutorial course called "Unity Essentials". You can find it on the Unity Learn website: https://learn.unity.com/. Here is the direct link to the course (find it on the Unity Learn website if the following link doesn't work): https://learn.unity.com/pathway/unity-essentials. This

by far was the best course for me as everything on YouTube by other creators was either outdated or didn't teach me the exact skills I needed. I also don't recommend skipping through sections of the course unless you really know those specific skills. The course has a lot of little tasks and tips throughout the tutorial that I found really useful.

4.) Note about Game Development Engines: I could've technically used any game development engine to pursue this project, however I found community resources for Unity to be widely accessible, hence why I picked Unity for this project.

References:

[1] “Virtualizing electrical appliances and gadgets input with human gestures – scientific figure,” ResearchGate. Accessed: May 29, 2026. [Online]. Available: https://www.researchgate.net/figure/Microsoft-Kinect-XBOX-360-gaming-console-sensor_fig1_276419008

[2] Reliance Digital, “Xbox 360 Kinect,” YouTube, Dec. 20, 2011. https://www.youtube.com/watch?v=oyjNqksc-m8 (accessed May 29, 2026).

[3] Innominate Games, “Kinect Sports: Table Tennis (HD),” *YouTube*. Jul. 09, 2020. [Online]. Available: https://www.youtube.com/watch?v=nCIKEeTXdz8

[4] Youtu.be, 2026. https://youtu.be/wgt0fNnyLLM (accessed May 29, 2026).

[5] Aneesh Pabolu, “GitHub - testUser453/VisionPong,” *GitHub*, 2026. https://github.com/testUser453/VisionPong (accessed May 29, 2026).